Université des Sciences et de la Technologie Houari Boumediene
Faculté d'Informatique

# TP ALGO3

---

## Support TP programmation C - les listes chainées -

---

Fait par :

**Nom et prénom : ABDELMALEK BENMEZIANE**

# Contents

**1 Les listes chaînées en C** **1**

1.1 Définition d'une liste chaînée . . . . . 1

1.2 Liste chaînée vs Tableau . . . . . 2

1.2.1 Conclusion . . . . . 2

1.3 Les types des listes chaînées . . . . . 2

1.4 Les opérations sur une liste chaînée . . . . . 2

1.5 Implémentation des opérations en langage C . . . . . 3

1.5.1 Liste chaînée simple . . . . . 3

1.5.2 Liste chaînée circulaire . . . . . 9

1.5.3 Liste chaînée double . . . . . 16

1.6 Conclusion . . . . . 23

# Chapter 1

# Les listes chaînées en C

## 1.1 Définition d'une liste chaînée

Une liste chaînée est une structure de données dynamique composée d'éléments appelés nœuds, chaque nœud contient :

- Une donnée (l'information à stocker)
- Un pointeur vers le nœud suivant dans la liste

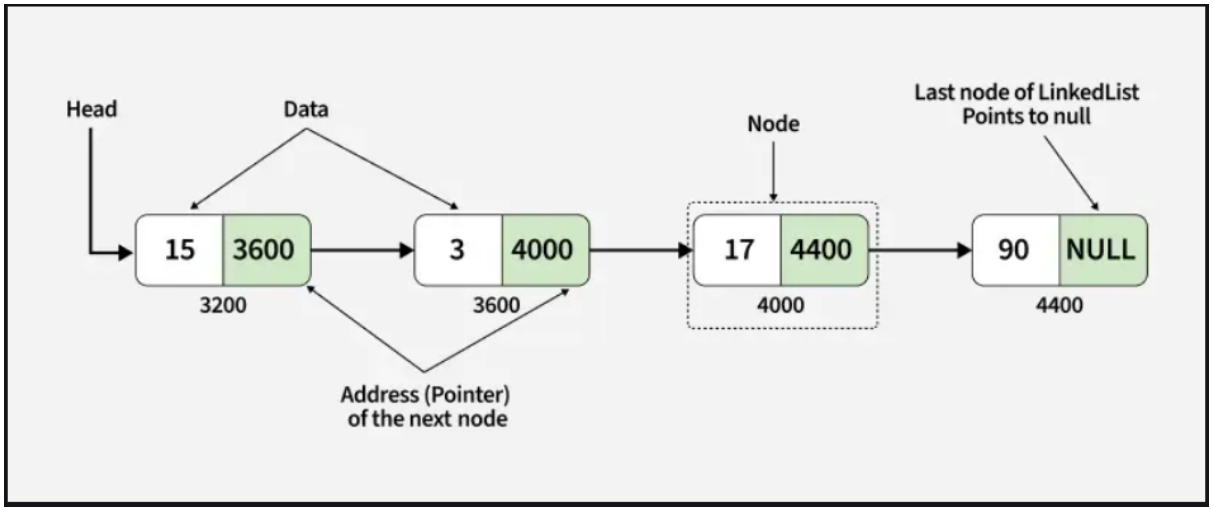


**Figure 1.1:** Linked list

Le premier nœud s'appelle la tête (head), et le dernier nœud pointe vers NULL, indiquant la fin de la liste.

## 1.2 Liste chaînée vs Tableau

| Critère | Liste chaînée | Tableau |
|---|---|---|
| **Taille** | Dynamique (s'adapte à l'exécution) | Fixe (déterminée à la compilation) |
| **Accès aux éléments** | Séquentiel (il faut parcourir la liste) | Direct par index (`tab[i]`) |
| **Insertion/Suppression** | Rapides (pas besoin de décaler les éléments) | Coûteuses (décalage nécessaire) |
| **Mémoire** | Occupation non contiguë | Contiguë |
| **Performance** | Moins efficace pour l'accès aléatoire | Plus efficace pour l'accès indexé |

**Table 1.1:** Comparaison entre une liste chaînée et un tableau en C

### 1.2.1 Conclusion

- Les listes chaînées sont idéales lorsque la taille des données varie souvent.
- Les tableaux sont préférables quand on connaît la taille fixe et qu'on a besoin d'un accès rapide par indice.

## 1.3 Les types des listes chaînées

- Les listes chaînées simples.
- Les listes chaînées circulaires.
- Les listes chaînées doubles.

## 1.4 Les opérations sur une liste chaînée

- Création d'une liste vide
- Insertion d'un élément (au début, à la fin ou à une position donnée)
- Suppression d'un élément (au début, à la fin ou à une position donnée)
- Parcours (afficher ou traiter chaque élément)
- Recherche d'un élément

- Calcul du nombre d'éléments de la liste

## 1.5 Implémentation des opérations en langage C

Les opérations sur les listes chaînées sont identiques pour tous les trois types avec des différences remarquées selon le besoin.

### 1.5.1 Liste chaînée simple

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int info;
    struct node *next;
} node;

void create(node **start, int data);
void display(node **start);
void insert_begin(node **start, int data);
void insert_end(node **start, int data);
void insert_pos(node **start, int pos, int data);
void delete_begin(node **start);
void delete_end(node **start);
void delete_pos(node **start, int pos);
void count(node **start);

int main() {
    node *start = NULL;

    // Cr er une liste avec des valeurs initiales
    create(&start, 10);
    create(&start, 20);
    create(&start, 30);
    printf("Initial list: ");
    display(&start);

    // Ins rer un   lment   au d but
    insert_begin(&start, 5);
```

```
    printf("After inserting 5 at the beginning: ");
    display(&start);

    // Ins rer un   lment      la fin
    insert_end(&start, 40);
    printf("After inserting 40 at the end: ");
    display(&start);

    // Ins rer un   lment      une position sp cifique
    insert_pos(&start, 2, 15); // Ins rer 15    la position 2
    printf("After inserting 15 at position 2: ");
    display(&start);

    // Supprimer le premier   lment
    delete_begin(&start);
    printf("After deleting the first element: ");
    display(&start);

    // Supprimer le dernier   lment
    delete_end(&start);
    printf("After deleting the last element: ");
    display(&start);


        // Supprimer un   lment      une position sp cifique
    delete_pos(&start, 2); // Supprimer l'  lment      la position 2
    printf("After deleting the element at position 2: ");
    display(&start);

    // Compter le nombre d'  lments   dans la liste
    count(&start);

    return 0;
}

void create(node **start, int data) {
    node *temp, *ptr;
    temp = (node *)malloc(sizeof(node));
    if (temp == NULL) {
```

```
        printf("Out of Memory Space\n");
        exit(0);
    }

    temp->info = data;
    temp->next = NULL;

    if (*start == NULL) {
        *start = temp;
    } else {
        ptr = *start;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        ptr->next = temp;
    }
}

void display(node **start) {
    node *ptr = *start;
    if (ptr == NULL) {
        printf("List is empty\n");
        return;
    }

    while (ptr != NULL) {
        printf("%d -> ", ptr->info);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

void insert_begin(node **start, int data) {
    node *temp = (node *)malloc(sizeof(node));
    if (temp == NULL) {
        printf("Out of Memory Space\n");
        return;
    }

```

```
    temp->info = data;
    temp->next = *start;
    *start = temp;
}

void insert_end(node **start, int data) {
    node *temp, *ptr;
    temp = (node *)malloc(sizeof(node));
    if (temp == NULL) {
        printf("Out of Memory Space\n");
        return;
    }

    temp->info = data;
    temp->next = NULL;

    if (*start == NULL) {
        *start = temp;
    } else {
        ptr = *start;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        ptr->next = temp;
    }
}

void insert_pos(node **start, int pos, int data) {
    node *ptr, *temp;
    int i;

    if (temp == NULL) {
        printf("Out of Memory Space\n");
        return;
    }

    if (pos == 1) {
        insert_begin(start,data);
    } else {
```

```

        temp = (node *)malloc(sizeof(node));
                temp->info = data;
                temp->next = NULL;
        ptr = *start;

        for (i = 1; i < pos - 1; i++) {
            ptr = ptr->next;
        }

        temp->next = ptr->next;
        ptr->next = temp;
    }
}

void delete_begin(node **start) {
    if (*start == NULL) {
        printf("List is Empty\n");
        return;
    }

    node *ptr = *start;
    *start = (*start)->next;
    free(ptr);
}

void delete_end(node **start) {
    if (*start == NULL) {
        printf("List is Empty\n");
        return;
    }

    node *ptr = *start, *temp = NULL;

    if (ptr->next == NULL) {
        free(ptr);
        *start = NULL;
    } else {
        while (ptr->next != NULL) {
```

```
            temp = ptr;
            ptr = ptr->next;
        }
        temp->next = NULL;
        free(ptr);
    }
}

void delete_pos(node **start, int pos) {
    int i;
    node *temp, *ptr;

    if (*start == NULL) {
        printf("The List is Empty\n");
        return;
    }

    if (pos == 1) {
        delete_begin(start);
    } else {
        ptr = *start;
        for (i = 0; i < pos - 1; i++) {
            temp = ptr;
            ptr = ptr->next;
        }
        temp->next = ptr->next;
        free(ptr);
    }
}

void count(node **start) {
    node *temp = *start;
    int cpt = 0;

    while (temp != NULL) {
        cpt++;
        temp = temp->next;
    }

```

```
    printf("The number of elements is: %d\n", cpt);
}
```

### 1.5.2 Liste chaînée circulaire

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node{
    int data;
    struct node* next;
}node;

void insertAtBeginning(node** head,int data);
void insertAtEnd(node** head,int data);
void insertAtPosition(node** head,int data,int pos);
void display(node** head);
void deleteAtBeginning(node** head);
void deleteAtEnd(node** head);
void deleteAtPosition(node** head,int pos);
void count(node** head);
int getch();


int main(){
        node *head=NULL;
        int choice,n,pos;
        while(1){
            system("cls");
            printf("---Circular linked list operations----\n");
            printf("1.Insert at the beginning\n");
            printf("2.Insert at the end\n");
            printf("3.Insert at specified position\n");
            printf("4.Delete from beginning\n");
            printf("5.Delete from the end\n");
            printf("6.Delete from specified position\n");
            printf("7.Display\n");
            printf("8.Count (number of elements)\n");
            printf("9.Exit\n");
```

```
        printf("----------------------------------------\n");
        printf("Enter your choice:");
        scanf("%d",&choice);
        switch(choice){
            case 1: printf("donner une valeur:");
                    scanf("%d",&n);
                    insertAtBeginning(&head,n);
                    break;

            case 2: printf("donner une valeur:");
                    scanf("%d",&n);
                    insertAtEnd(&head,n);
                    break;

            case 3: printf("donner une valeur:");
                    scanf("%d",&n);
                    printf("donner un position:");
                    scanf("%d",&pos);
                    insertAtPosition(&head,n,pos);
                    break;

            case 4: deleteAtBeginning(&head);
                    break;

            case 5: deleteAtEnd(&head);
                    break;

            case 6: printf("donner un position:");
                    scanf("%d",&pos);
                    deleteAtPosition(&head,pos);
                    break;

            case 7: display(&head);
                    break;

            case 8: count(&head);
                    break;

            case 9:exit(0);
```

```
                        break;

                default: printf("Wrong Choice\n");
                        break;
            }

            getch();
        }

    return 0;
}

void insertAtBeginning(node** head,int data){

        node *newNode = (node*) malloc(sizeof(node));
        newNode->data = data;

        if(*head == NULL){
            *head = newNode;
            (*head)->next = *head;
            return;
        }

        node* curr = *head;

        while(curr->next != *head){
            curr = curr->next;
        }

        // assign LL's last node's next as this new node
        curr->next = newNode;
        // assign newNode's next as current head
        newNode->next = *head;
        // change head to this new node
        *head = newNode;
}
```

```c
void insertAtEnd(node** head,int data){

        node *newNode = (node*) malloc(sizeof(node));
        newNode->data = data;

        if(*head == NULL){
            *head = newNode;
            (*head)->next = *head;
            return;
        }

        node *curr = *head;

        while(curr->next != *head){
            curr = curr->next;
        }

    // assign LL's current last node's next as this new node
        curr->next = newNode;
    // assign this new node's next as current head of LL
        newNode->next = *head;
}

void insertAtPosition(node** head,int data,int pos){

        node *newnode, *curNode;
        int i;

        if(*head == NULL){
            printf ("List is empty");
        }

        if (pos == 1){
            insertAtBeginning(head, data);
            return;
        }else{
            newnode = (node*)malloc(sizeof(node));
            newnode->data = data;
            curNode = *head;
```

```
            while (--pos > 1){
                  curNode = curNode->next;
            }

            newnode->next = curNode->next;
            curNode->next = newnode;
        }
}

void display(node** head){

        if(*head == NULL){
                return;
        }

        node* temp = *head;

        do{
            printf ("%d->", temp->data);
            temp = temp->next;
        }while (temp != *head);

        printf ("NULL\n");
}

void deleteAtBeginning(node** head){

        node *tempNode = *head;

        if(*head == NULL){
            printf ("Linked List Empty, nothing to delete");
            return;
        }

        if(tempNode->next == *head){
            *head = NULL;
            return;
        }

```

```
        node *curr = *head;

        while (curr->next != *head){
                curr = curr->next;
        }

      // assign last node's next to 2nd node in CLL
        curr->next = (*head)->next;
        *head = (*head)->next; // move head to next node
        free (tempNode);
}

void deleteAtEnd(node** head){

        node *tempNode = *head;
        node *previous;

        if(*head == NULL){
            printf ("Linked List Empty, nothing to delete");
            return;
        }

        if (tempNode->next == *head){
            *head = NULL;
            return;
        }

        while (tempNode->next != *head){
  // store previous link node as we need to change its next val
            previous = tempNode;
            tempNode = tempNode->next;
        }

        // Curr assign 2nd last node's next to head
        previous->next = *head;
        free (tempNode); // delete the last node
}


```

```c
void deleteAtPosition(node** head,int pos){

    int i;
    node *temp,*ptr;

    if(*head==NULL){
        printf("The List is Empty\n");
        exit(0);
    }else{
        if(pos==1){
            deleteAtBeginning(head);
        }else{
            ptr=*head;
            for(i=0;i<pos-1;i++) {
                temp=ptr;
                ptr=ptr->next;
                if(ptr==NULL){
                    printf("Position not Found\n");
                    return;
                }
            }

            temp->next = ptr->next;
            printf("The deleted element is:%d\n",ptr->data);
            free(ptr);
        }
    }
}

void count(node** head){
        node* temp = *head;
        int cpt = 1;

        while(temp->next!=*head){
        temp = temp->next;
        cpt++;
        }
        printf("Le nombre d'elements est: %d",cpt);
}
```

### 1.5.3 Liste chaînée double

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int data;
    struct node* prev;
    struct node* next;
} node;

void insertAtBeginning(node** head, node** tail, int data);
void insertAtEnd(node** head, node** tail, int data);
void insertAtPosition(node** head, node** tail, int data, int pos);
void deleteAtBeginning(node** head, node** tail);
void deleteAtEnd(node** head, node** tail);
void deleteAtPosition(node** head, node** tail, int pos);
void display(node** head, node** tail);
void count(node** head);
int getch();


int main() {

        node* head = NULL;
        node* tail = NULL;

        int choice,n,pos;
        while(1){
            system("cls");
            printf("---Doubly linked list operations---\n");
            printf("1.Insert at the beginning\n");
            printf("2.Insert at the end\n");
            printf("3.Insert at specified position\n");
            printf("4.Delete from beginning\n");
            printf("5.Delete from the end\n");
            printf("6.Delete from specified position\n");
            printf("7.Display\n");
            printf("8.Count (number of elements)\n");
            printf("9.Exit\n");
```

```
        printf("----------------------------------------\n");
        printf("Enter your choice:");
        scanf("%d",&choice);
        switch(choice){
            case 1: printf("donner une valeur:");
                    scanf("%d",&n);
                    insertAtBeginning(&head,&tail,n);
                    break;

            case 2: printf("donner une valeur:");
                    scanf("%d",&n);
                    insertAtEnd(&head,&tail,n);
                    break;

            case 3: printf("donner une valeur:");
                    scanf("%d",&n);
                    printf("donner un position:");
                    scanf("%d",&pos);
                    insertAtPosition(&head,&tail,n,pos);
                    break;

            case 4: deleteAtBeginning(&head,&tail);
                    break;

            case 5: deleteAtEnd(&head,&tail);
                    break;

            case 6: printf("donner un position:");
                    scanf("%d",&pos);
                    deleteAtPosition(&head,&tail,pos);
                    break;

            case 7: display(&head,&tail);
                    break;

            case 8: count(&head);
                    break;

            case 9:exit(0);
```

```
                        break;

                default: printf("Wrong Choice\n");
                        break;
            }

            getch();
        }

    return 0;
}

void insertAtBeginning(node** head, node** tail, int data) {

    node* newNode = (node*)malloc(sizeof(node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        *tail = newNode;
    } else {
        newNode->next = *head;
        (*head)->prev = newNode;
        *head = newNode;
    }
}

void insertAtEnd(node** head, node** tail, int data) {

    node* newNode = (node*)malloc(sizeof(node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        *tail = newNode;
```

```
    } else {
        (*tail)->next = newNode;
        newNode->prev = *tail;
        *tail = newNode;
    }
}

void insertAtPosition(node** head, node** tail, int data, int pos){

    if (pos <= 0) {
        printf("Invalid position\n");
        return;
    }

    if (pos == 1) {
        insertAtBeginning(head, tail, data);
        return;
    }

    node* newNode = (node*)malloc(sizeof(node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;

    node* temp = *head;
    int count = 1;
    while (temp != NULL && count < pos - 1) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL) {
        printf("Invalid position\n");
        return;
    }

    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL) {
```

```
        temp->next->prev = newNode;
    }
    temp->next = newNode;

    if (newNode->next == NULL) {
        *tail = newNode;
    }
}

void deleteAtBeginning(node** head, node** tail) {

    if (*head == NULL) {
        printf("List is empty.\n");
        return;
    }

    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        *tail = NULL;
        return;
    }

    node* temp = *head;
    *head = temp->next;
    (*head)->prev = NULL;
    free(temp);
}

void deleteAtEnd(node** head, node** tail) {

    if (*head == NULL) {
        printf("List is empty.\n");
        return;
    }

    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
```

```
        *tail = NULL;
        return;
    }

    node* temp = *tail;
    *tail = temp->prev;
    (*tail)->next = NULL;
    free(temp);
}

void deleteAtPosition(node** head, node** tail, int pos) {

    if (pos <= 0) {
        printf("Invalid position.\n");
        return;
    }

    if (*head == NULL) {
        printf("List is empty.\n");
        return;
    }

    if (pos == 1) {
        deleteAtBeginning(head, tail);
        return;
    }

    node* current = *head;
    int count = 1;

    while (current != NULL && count < pos) {
        current = current->next;
        count++;
    }

    if (current == NULL) {
        printf("Invalid position.\n");
        return;
    }
```

```

    if (current->next == NULL) {
        *tail = current->prev;
        (*tail)->next = NULL;
        free(current);
        return;
    }

    current->prev->next = current->next;
    current->next->prev = current->prev;
    free(current);
}

void display(node** head, node** tail) {

    if (*head == NULL) {
        printf("List is empty.\n");
        return;
    }

    node* temp = *head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d->", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

void count(node** head) {
    int cpt = 0;
    node* temp = *head;

    while (temp != NULL) {
        cpt++;
        temp = temp->next;
    }
    printf("Number of nodes: %d\n", cpt);
}
```

## 1.6 Conclusion

Les listes chaînées constituent une structure de données fondamentale en langage C, illustrant parfaitement la gestion dynamique de la mémoire et la manipulation des pointeurs. Contrairement aux tableaux statiques, elles permettent une allocation flexible, facilitant les opérations d'insertion et de suppression sans contrainte de taille.

Cependant, cette flexibilité a un coût : les listes chaînées demandent un accès séquentiel aux éléments et une gestion explicite de la mémoire, ce qui peut rendre leur utilisation plus complexe et moins efficace pour certaines opérations.

En résumé, le choix entre un tableau et une liste chaînée dépend du contexte :

- Si la taille des données est fixe et l'accès rapide est prioritaire → Tableau.
- Si la taille varie et que les insertions/suppressions sont fréquentes → Liste chaînée.

Ainsi, la maîtrise des listes chaînées est une étape essentielle pour comprendre les structures de données dynamiques, les pointeurs, et plus largement, la logique de bas niveau du langage C.